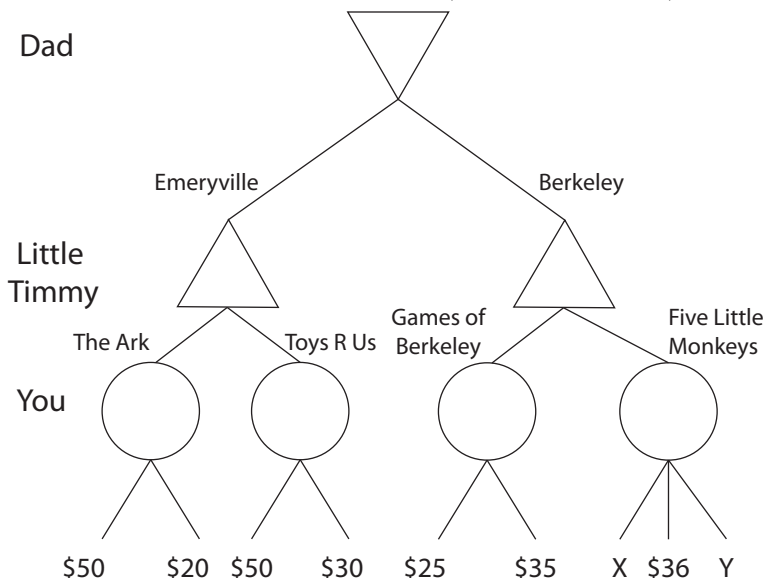


Q2. [9 pts] Expectimax

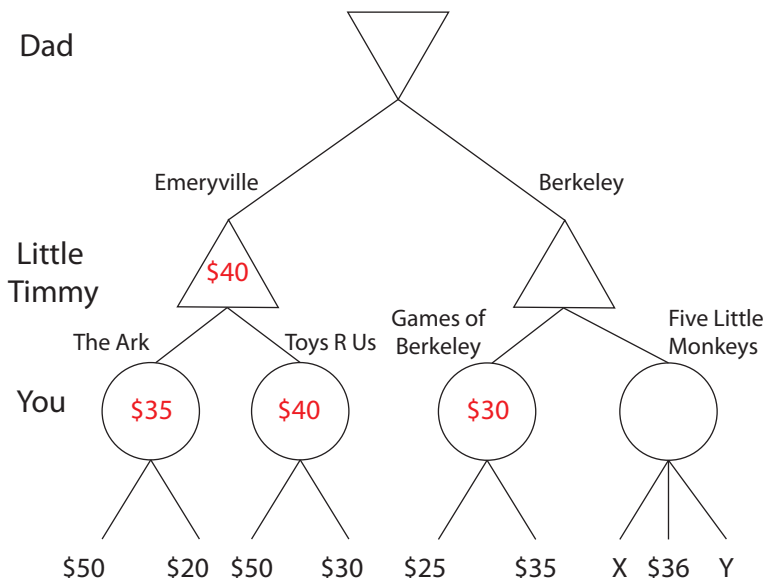
Your little brother Timmy has a birthday and he was promised a toy. However, Timmy has been misbehaving lately and Dad thinks he deserves the least expensive present. Timmy, of course, wants the most expensive toy. Dad will pick the city from which to buy the toy, Timmy will pick the store and you get to pick the toy itself. You don't want to take sides so you decide to pick a toy at random. All prices (including X and Y) are assumed to be nonnegative.



- (a) [1 pt] Fill in the values of all the nodes that don't depend on X or Y.
- (b) [3 pts] What values of X will make Dad pick Emeryville regardless of the price of Y?
- (c) [3 pts] We know that Y is at most \$30. What values of X will result in a toy from Games of Berkeley regardless of the exact price of Y?
- (d) [2 pts] Normally, alpha-beta pruning is not used with expectimax. However, with some additional information, it is possible to do something similar. Which **one** of the following conditions on a problem are required to perform pruning with expectimax?
1. The children of the expectation node are leaves.
 2. All values are positive.
 3. The children of the expectation node have specified ranges.
 4. The child to prune is last.

Q2. [9 pts] Expectimax

Your little brother Timmy has a birthday and he was promised a toy. However, Timmy has been misbehaving lately and Dad thinks he deserves the least expensive present. Timmy, of course, wants the most expensive toy. Dad will pick the city from which to buy the toy, Timmy will pick the store and you get to pick the toy itself. You don't want to take sides so you decide to pick a toy at random. All prices (including X and Y) are assumed to be nonnegative.



- (a) [1 pt] Fill in the values of all the nodes that don't depend on X or Y.
- (b) [3 pts] What values of X will make Dad pick Emeryville regardless of the price of Y?

Dad will pick Emeryville if the value of the Berkeley node is more than the value of the Emeryville node (\$40), that is if:

$$\frac{x + y + 36}{3} > 40 \Leftrightarrow x > 84 - y \Leftrightarrow x > 84$$

.

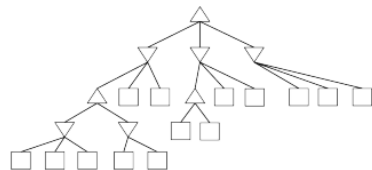
- (c) [3 pts] We know that Y is at most \$30. What values of X will result in a toy from Games of Berkeley regardless of the exact price of Y?

Games of Berkeley will be chosen if the value of the "Five Little Monkeys" node is less than the value of the "Games of Berkeley" node (\$30):

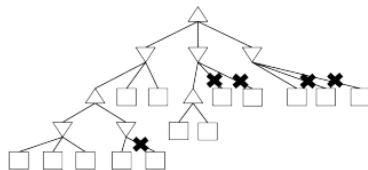
$$\frac{x + y + 36}{3} < 30 \Leftrightarrow x < 54 - y \Leftrightarrow x < 24$$

(g) Assume we run $\alpha - \beta$ pruning expanding successors from left to right on a game with tree as shown in Figure 1 (a). Then we have that:

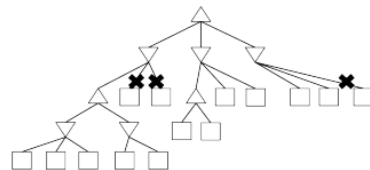
- (i) [rue or false] For some choice of pay-off values, no pruning will be achieved (shown in Figure 1 (a)).
- (ii) [rue or false] For some choice of pay-off values, the pruning shown in Figure 1 (b) will be achieved.
- (iii) [rue or false] For some choice of pay-off values, the pruning shown in Figure 1 (c) will be achieved.
- (iv) [rue or false] For some choice of pay-off values, the pruning shown in Figure 1 (d) will be achieved.
- (v) [rue or false] For some choice of pay-off values, the pruning shown in Figure 1 (e) will be achieved.
- (vi) [rue or false] For some choice of pay-off values, the pruning shown in Figure 1 (f) will be achieved.



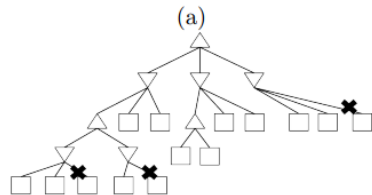
(a)



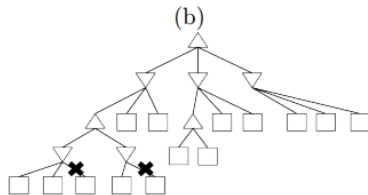
(b)



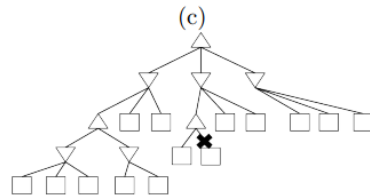
(c)



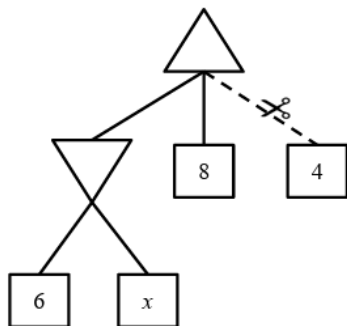
(d)



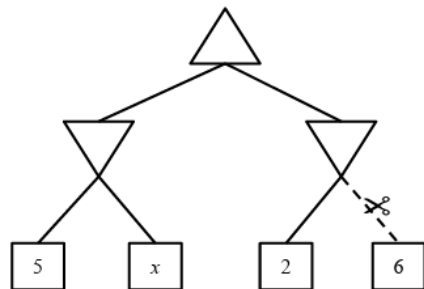
(e)



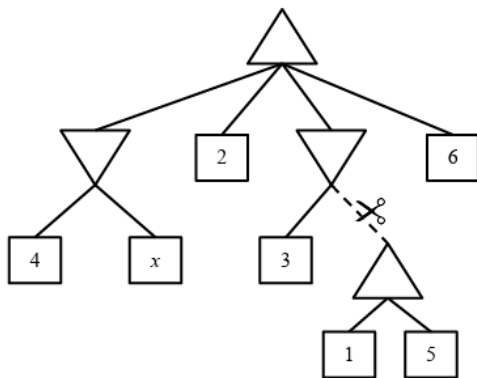
(f)



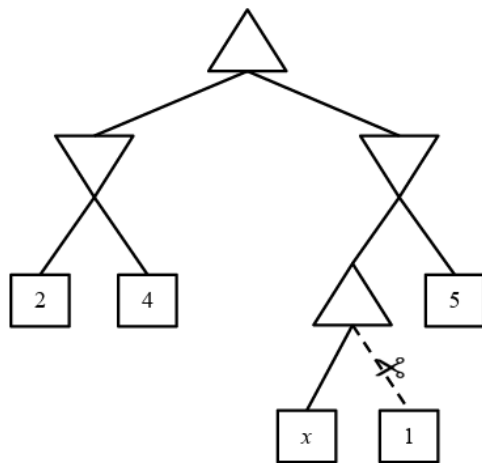
(b) Tree 1. Answer: _____



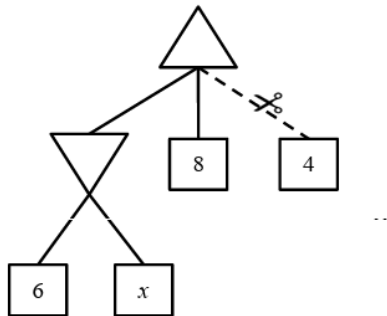
(c) Tree 2. Answer: _____



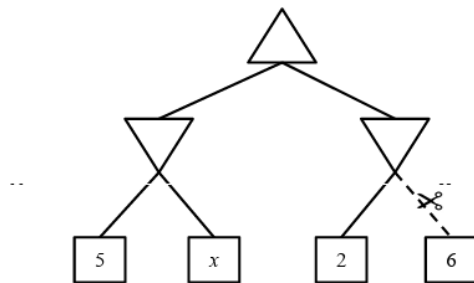
(d) Tree 3. Answer: _____



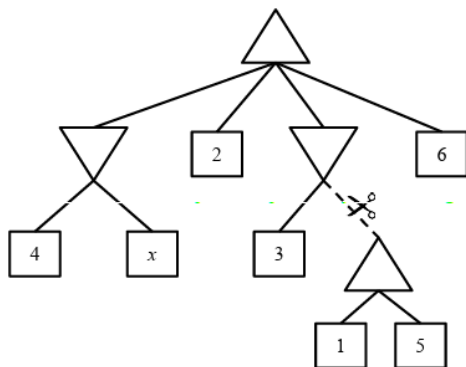
(e) Tree 4. Answer: _____



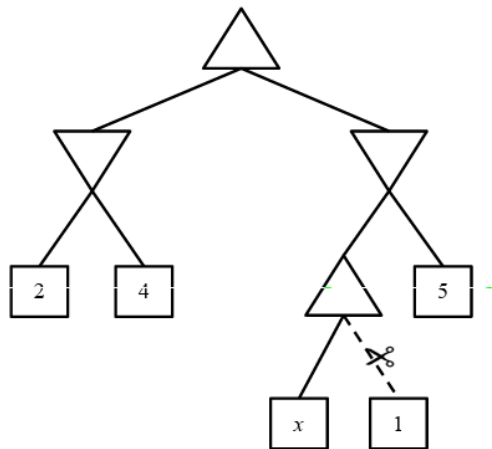
(b) Tree 1. Answer: None



(c) Tree 2. Answer: $x \geq 2$



(d) Tree 3. Answer: $x \geq 3$

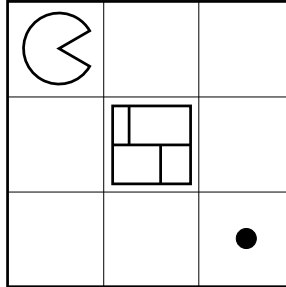


(e) Tree 4. Answer: None

5. (24 points) Surrealist Pacman

In the game of Surrealist Pacman, Pacman $\ominus$ plays against a moving wall $\boxplus$. On Pacman's turn, Pacman must move in one of the four cardinal directions, and must move into an unoccupied square. On the wall's turn, the wall must move in one of the four cardinal directions, and must move into an unoccupied square. The wall cannot move into a dot-containing square. Staying still is not allowed by either player. Pacman's score is always equal to the number of dots he has eaten.

The first game begins in the configuration shown below. Pacman moves first.



(a) (2 pt) Draw a game tree with one move for each player. Draw only the legal moves.

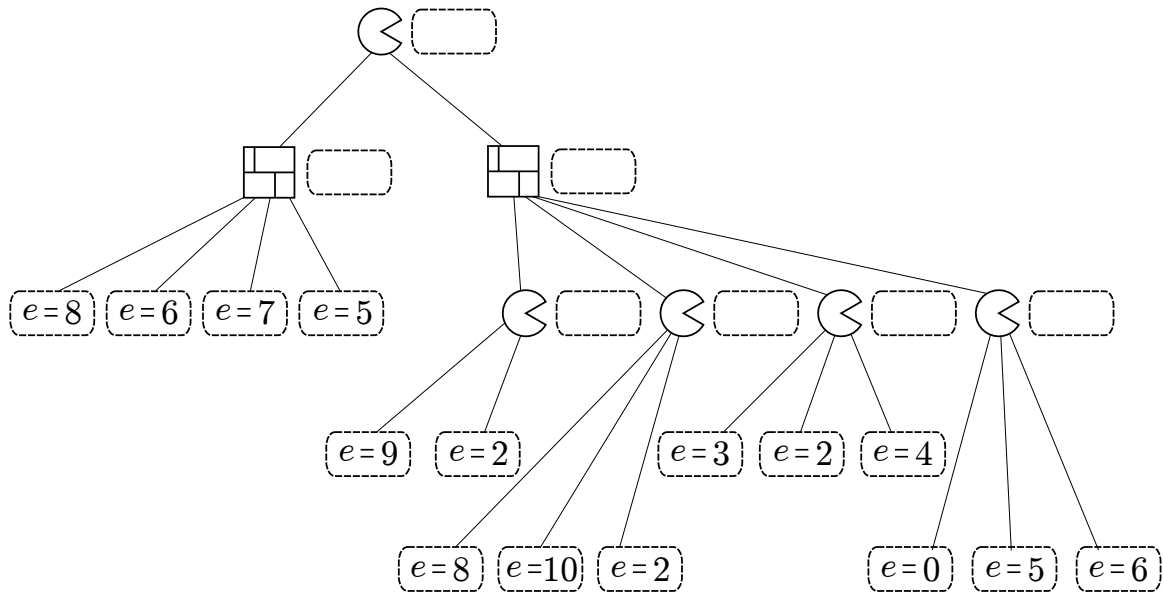
(b) (1 pt) According to the depth-limited game tree you drew above what is the value of the game? Use Pacman's score as your evaluation function.

(c) (1 pt) If we were to consider a game tree with ten moves for each player (rather than just one), what would be the value of the game as computed by minimax?

A second game is played on a more complicated board. A partial game tree is drawn, and leaf nodes have been scored using an (unknown) evaluation function e .

(d) (3 pt) In the dashed boxes, fill in the values of all internal nodes using the minimax algorithm.

(e) (3 pt) Cross off any nodes that are not evaluated when using alpha-beta pruning (assuming the standard left-to-right traversal of the tree).



Suppose that this evaluation function has a special property: it is known to give the correct minimax value of any internal node to within 2, and the correct minimax values of the leaf nodes exactly. That is, if v is the true minimax value of a particular node, and e is the value of the evaluation function applied to that node, $e - 2 \leq v \leq e + 2$, and $v = e$ if the node is a dashed box in the tree below.

Using this special property, you can modify the alpha-beta pruning algorithm to prune more nodes.

- (f) (10 pt) Standard alpha-beta pseudocode is given below (only the max-value recursion). Fill in the boxes on the right to replace the corresponding boxes on the left so that the pseudocode prunes as many nodes as possible, taking account of this special property of the evaluation function.

```

function MAX-VALUE(node,  $\alpha$ ,  $\beta$ )
   $e \leftarrow$  EVALUATIONFUNCTION(node)
  if node is leaf then
    return  $e$ 
  end if (1)
   $v \leftarrow -\infty$ 
  for child  $\leftarrow$  CHILDREN(node) do

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\textit{child}, \alpha, \beta))$

 (2)
    if  $v \geq \beta$  then
      return  $v$ 
    end if
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  end for
  return  $v$ 
end function

```

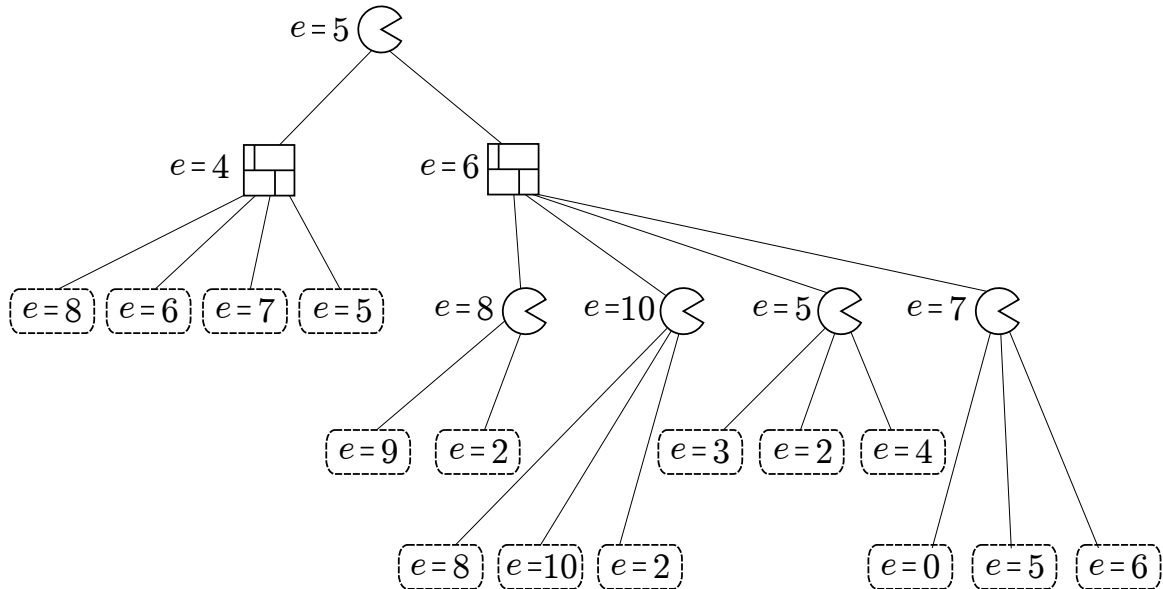
Fill in these boxes:

(1)

(2)

The same game tree is shown below, with the evaluation function applied to *internal* as well as leaf nodes.

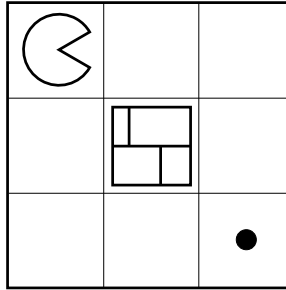
- (g) (4 pt) In the game tree below cross off any nodes that can be pruned assuming the special property holds true. If you are not sure you correctly formalized into pseudo-code your intuition on how to exploit the special property for improved pruning, make sure to annotate your pruned nodes with a brief explanation of why each of them was pruned.



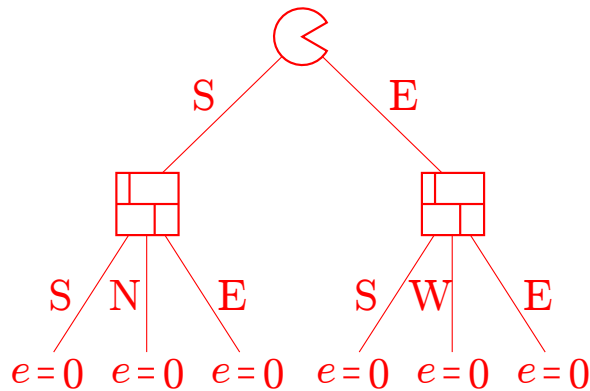
5. (24 points) Surrealist Pacman

In the game of Surrealist Pacman, Pacman  plays against a moving wall . On Pacman's turn, Pacman must move in one of the four cardinal directions, and must move into an unoccupied square. On the wall's turn, the wall must move in one of the four cardinal directions, and must move into an unoccupied square. The wall cannot move into a dot-containing square. Staying still is not allowed by either player. Pacman's score is always equal to the number of dots he has eaten.

The first game begins in the configuration shown below. Pacman moves first.



- (a) (2 pt) Draw a game tree with one move for each player. Draw only the legal moves.



- (b) (1 pt) According to the depth-limited game tree you drew above what is the value of the game? Use Pacman's score as your evaluation function.

0, since all leaves have value 0 as Pacman cannot eat a dot in one move.

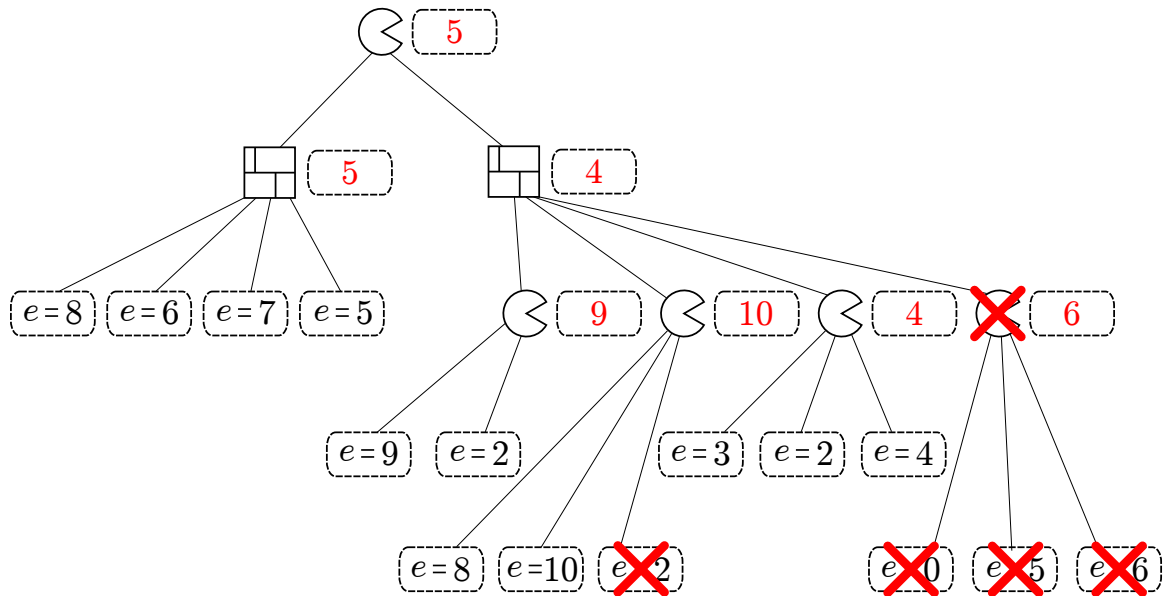
- (c) (1 pt) If we were to consider a game tree with ten moves for each player (rather than just one), what would be the value of the game as computed by minimax?

1. Pacman can force a win and eat the dot in ten moves, so the score increases to 1 once the dot is eaten and remains there. For an example of such a win, consider the sequences E-S-S-E or E-S-E-S by Pacman and the movement rules of the players.

A second game is played on a more complicated board. A partial game tree is drawn, and leaf nodes have been scored using an (unknown) evaluation function e .

(d) (3 pt) In the dashed boxes, fill in the values of all internal nodes using the minimax algorithm.

(e) (3 pt) Cross off any nodes that are not evaluated when using alpha-beta pruning (assuming the standard left-to-right traversal of the tree).



Running alpha-beta pruning on the game tree.

Root: $\alpha = -\infty, \beta = \infty$

-- Left wall: $\alpha = -\infty, \beta = \infty$

---- Leaf node: $e = 8$. Propagate $e = 8$ back to parent.

-- Left wall: Current value is 8. $\alpha = -\infty, \beta = 8$. Max doesn't have a best value, so continue exploring.

---- Leaf node: $e = 6$. Propagate $e = 6$ back to parent.

-- Left wall: Current value is 6. $\alpha = -\infty, \beta = 6$. Max doesn't have a best value, so continue exploring.

---- Leaf node: $e = 7$. Propagate $e = 7$ back to parent.

-- Left wall: No update. Current value is 6. $\alpha = -\infty, \beta = 6$. Max doesn't have a best value, so continue exploring.

---- Leaf node: $e = 5$. Propagate $e = 5$ back to parent.

-- Left wall: Current value is 5. We're done here, so propagate 5 to root.

Root: Current value is 5. $\alpha = 5, \beta = \infty$. Explore right.

-- Right wall: $\alpha = 5, \beta = \infty$.

---- 1st Pac: $\alpha = 5, \beta = \infty$

----- Leaf node: $e = 9$. Propagate $e = 9$ back to parent.

---- 1st Pac: Current value is 9. $\alpha = 9, \beta = \infty$ MIN doesn't have a best value, so continue exploring.

----- Leaf node: $e = 2$. Propagate $e = 2$ back to parent.

---- 1st Pac: No change. Current value is 9. Propagate 9 to parent.

-- Right wall: Current value is now 9. $\alpha = 5, \beta = 9$. MIN wants anything less than 9 at this point, but it's still possible for MAX to get more than 5. Continue exploring.

---- 2nd Pac: $\alpha = 5, \beta = 9$

----- Leaf node: $e = 8$. Propagate $e = 8$ back to parent.

---- 2nd Pac: Current value is now 8. $\alpha = 8, \beta = 9$. Again, still possible for both players to benefit (Imagine value = 8.5). Continue exploring.

- - - - - Leaf node: $e = 10$. Propagate $e = 10$ back to parent.
 - - - - 2nd Pac: Current value is now 10. So now, we know that $v > \beta$, which means that one of the players is going to be unhappy. MAX wants something more than 10, but MIN is only satisfied with something less than 9, so we don't have to keep exploring.
 - - - - *PRUNE* $e = 2$.
 - - - - 2nd Pac: returns value of 10 to parent.
 - - Left Wall: No change in value, current value is still 9. $\alpha = 5, \beta = 9$. Again, still possible for both players to benefit, so continue exploring.
 - - - - 3rd Pac: $\alpha = 5, \beta = 9$
 - - - - - Leaf node: $e = 3$. Propagate $e = 3$ back to parent.
 - - - - 3rd Pac: Current value is 3. $\alpha = 5, \beta = 9$. Continue exploring.
 - - - - - Leaf node: $e = 2$. Propagate $e = 2$ back to parent.
 - - - - 3rd Pac: No change in value. Current value is 3. $\alpha = 5, \beta = 9$. Continue exploring.
 - - - - - Leaf node: $e = 4$. Propagate $e = 4$ back to parent.
 - - - - 3rd Pac: Current value is 4. We're done, so return value of 4 to parent.
 - - Left Wall: Current value becomes 4. At this point, we know that MIN wants anything that is less than or equal to 4. However, MAX is only satisfied with something that is 5 or greater. Hence, we don't need to explore the rest of the children of this node since MAX will never let the game get down to this branch.- -

Prune rest

(Filling values returned by alpha-beta or not crossing off children of a crossed off node were not penalized.)

Suppose that this evaluation function has a special property: it is known to give the correct minimax value of any internal node to within 2, and the correct minimax values of the leaf nodes exactly. That is, if v is the true minimax value of a particular node, and e is the value of the evaluation function applied to that node, $e - 2 \leq v \leq e + 2$, and $v = e$ if the node is a dashed box in the tree below.

Using this special property, you can modify the alpha-beta pruning algorithm to prune more nodes.

- (f) (10 pt) Standard alpha-beta pseudocode is given below (only the max-value recursion). Fill in the boxes on the right to replace the corresponding boxes on the left so that the pseudocode prunes as many nodes as possible, taking account of this special property of the evaluation function.

```

function MAX-VALUE(node,  $\alpha$ ,  $\beta$ )
   $e \leftarrow$  EVALUATIONFUNCTION(node)
  if node is leaf then
    return  $e$ 
  end if (1)
   $v \leftarrow -\infty$ 
  for child  $\leftarrow$  CHILDREN(node) do

$v \leftarrow$  MAX( $v$ , MIN-VALUE(child,  $\alpha$ ,  $\beta$ )) (2)

if  $v \geq \beta$  then
      return  $v$ 
    end if
     $\alpha \leftarrow$  MAX( $\alpha$ ,  $v$ )
  end for
  return  $v$ 
end function

```

Fill in these boxes:

(1)

if $e - 2 \geq \beta$ **then**
 return $e - 2$
end if

(2)

$v \leftarrow$ MAX(v , MIN-VALUE(*child*,
 MAX(α , $e - 2$), MIN(β , $e + 2$)))

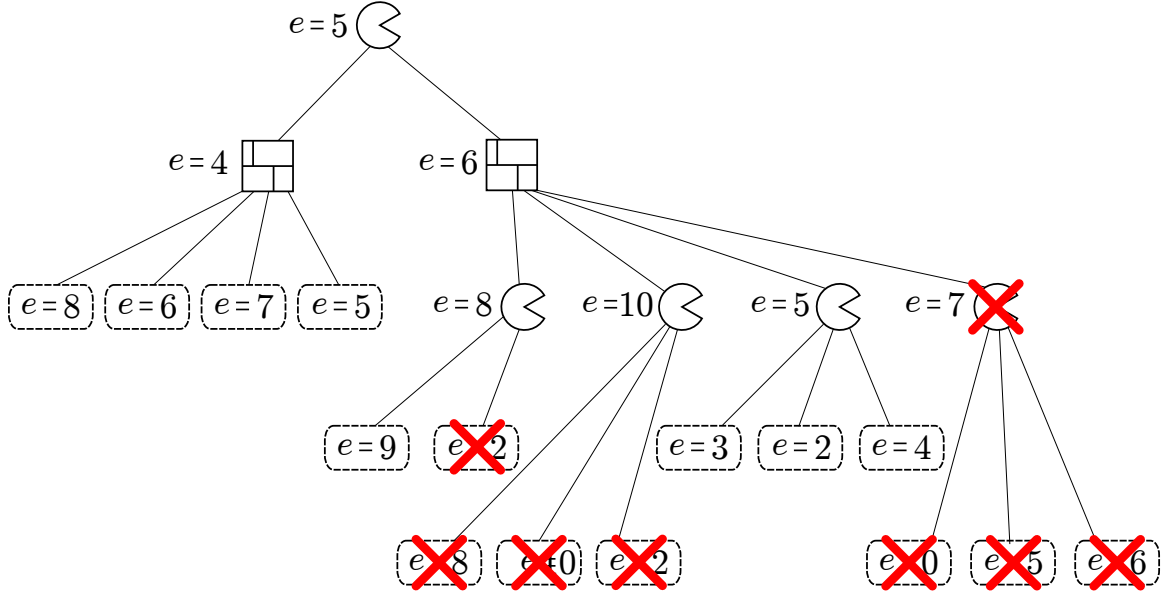
For (1), the special property guarantees $e - 2 \geq v$ and $e - 2$ is the smallest such value, so v is substituted out without violating the β inequality.

For (2), both sides of the inequality for the special property are used to bound the best action value for MAX α and the best action value for MIN β . α and β are sharpened whenever the ± 2 guarantee on e gives a higher lower-bound (α) or lower upper-bound (β).

(Variations are possible.)

The same game tree is shown below, with the evaluation function applied to *internal* as well as leaf nodes.

- (g) (4 pt) In the game tree below cross off any nodes that can be pruned assuming the special property holds true. If you are not sure you correctly formalized into pseudo-code your intuition on how to exploit the special property for improved pruning, make sure to annotate your pruned nodes with a brief explanation of why each of them was pruned.



Running pruning on game tree in detail. W1 and W2 refer to the left and right wall nodes respectively. P1, P2, P3, P4 refer to Pacman nodes on the third level, left to right.

Root: Evaluation function returns 5. Range of value: $[3, 7]$. Set $\alpha : 3, \beta : 7$ and $\text{explore}(W1, \alpha, \beta)$.

- W1: Evaluation function returns 4. Range of value: $[2, 6]$. Set $\alpha : 3, \beta : 6$.

- - Leaf node: $e = 8$. Send 8 back to W1.

- W1: $v = 8$. $v < \alpha$? No. This means that MAX might still prefer this path.

- - Leaf node: $e = 6$. Send 6 back to W1.

- W1: $6 < 8$ so $v = 6$. $v < \alpha$? No. Continue exploring.

- - Leaf node: $e = 7$. Send 7 back to W1.

- W1: $7 > 6$, so no change, $v = 6$. $v < \alpha$? No. Continue exploring.

- - Leaf node: $e = 5$. Send 5 back to W1.

- W1: $5 < 6$, so $v = 5$. Done. Send 5 back to root.

Root: Gets value 5, so $v = 5$. $\alpha : \max(3, 5) = 5, \beta : 7$. Still exploring right branch since MAX could get a value $5 \leq v \leq 7$. $\text{Explore}(W2, \alpha, \beta)$.

- W2: Evaluation function returns 6. Range of values $[4, 8]$. $\alpha : 5, \beta : 7$, $\text{explore}(P1, \alpha, \beta)$.

- - P1: Evaluation function returns 8. Range: $[6, 10]$, $\alpha : 6, \beta : 7$.

- - - Leaf node: $e = 9$. Send $e = 9$ back to P1.

- - P1: $v = 9$. $v > \beta$? Yes!. We can prune here since P1 wants any value > 9 . However, at the root we know that the maximum value that MAX can get is 7. Hence, there is no way the game can get down to P1. (Meaning that the value at the root can not be 9).

- - - Leaf node: Prune $e = 2$. Return 9 to W2.

- W2: $v = 9$. α, β don't change: $\alpha : 5, \beta : 7$. $\text{Explore}(P2, \alpha, \beta)$.

- - P2: Evaluation function returns 10. Range $[8, 12]$, $\alpha : 8, \beta : 7$. Notice that the best value for MIN that can be achieved at P2 is 8. However, the best value for MIN at the root is 7. Hence, there's no way the game can get down to P2. (Meaning the value of the root can not be 8). *Prune all of P2's children!*. We can return 8 to W2 since we know that there is some other path through W2 that yields a reward ≤ 7 .

- W2: $v : \min(8, 9) = 8, \alpha : 5, \beta : 7. v < \alpha?$ No! Explore(P3, α, β).
- - P3: Evaluation function returns 5. Range: $[3, 7], \alpha : 5, \beta : 7$.
- - - Leaf node: $e = 5$. Send 3 back to P3.
- - P3: $v = 3. v > \beta?$ No! Meaning MIN might still prefer this branch. $\alpha : 5, \beta : 7$.
- - - Leaf node: $e = 2$. Send 2 back to P3.
- - P3: $v = 3. v > \beta?$ No! $\alpha : 5, \beta : 7$.
- - - Leaf node: $e = 4$. Send 4 back to P3.
- - P3: $v = 4$. Done. Return 4 to W2.
- W2: $v : \min(8, 4) = 4, \alpha : 5, \beta : 7. v < \alpha?$ Yes! Since MAX can guarantee a value of 5 and MIN will only accept something < 4 , don't need to explore any further. Prune P_4 and all its children. Return 4 to root.
Root: Done.

Q5. [9 pts] Games: Three-Player Cookie Pruning

Three of your TAs, Alvin, Sergey, and James rent a cookie shuffler, which takes in a set number of cookies and groups them into 3 batches, one for each player. The cookie shuffler has three levers (with positions either UP or DOWN), which act to control how the cookies are distributed among the three players. Assume that 30 cookies are initially put into the shuffler.

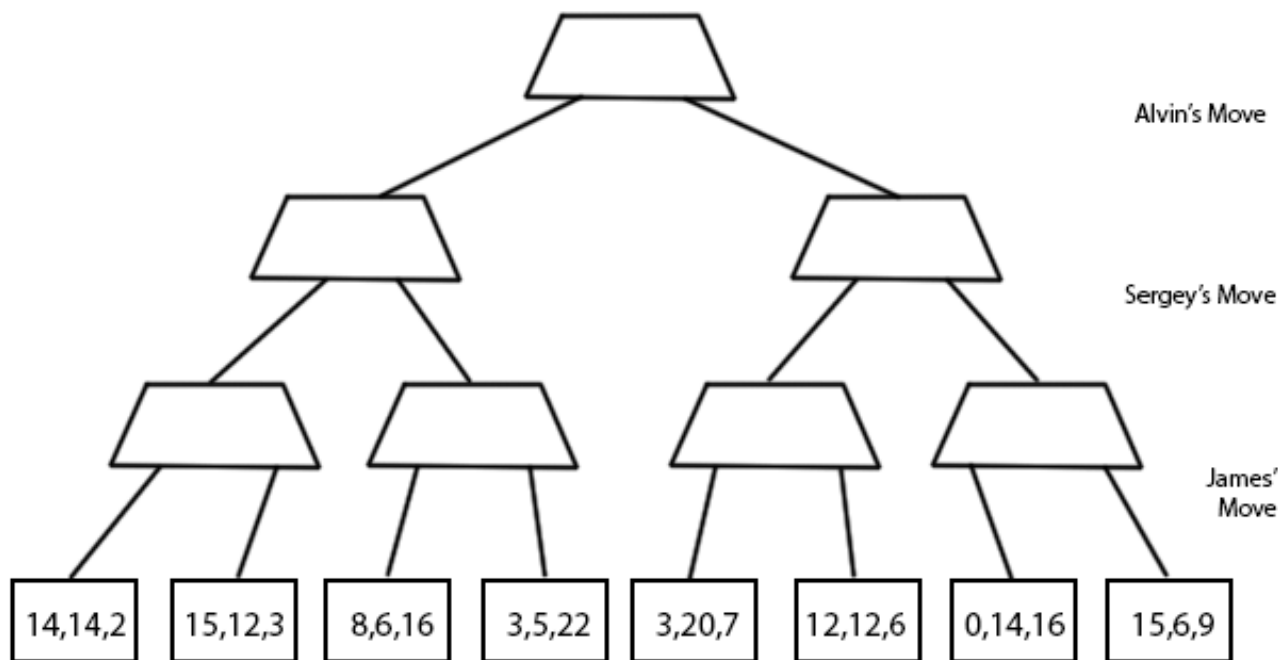
Each player controls one lever, and they act in turn. Alvin goes first, followed by Sergey, and finally James. Assume that all players are able to calculate the payoffs for every player at the terminal nodes. Assume the payoffs at the leaves correspond to the number of cookies for each player in their corresponding turn order. Hence, an utility of (7,10,13) corresponds to Alvin getting 7 cookies, Sergey getting 10 cookies, and James getting 13 cookies. No cookies are lost in the process, so the sum of cookies of all three players must equal the number of cookies put into the shuffler. Players want to maximize their own number of cookies.

(a) [3 pts] **What is the utility triple propagated up to the root?**

(b) [6 pts] **Is pruning possible in this game? Fill in "Yes" or "No". If yes, cross out all nodes (both leaves and intermediate nodes) that get pruned. If no, explain in one sentence why pruning is not possible.** Assume the tree traversal goes from left to right.

☐ Yes.

☐ No, Reasoning:



Q5. [9 pts] Games: Three-Player Cookie Pruning

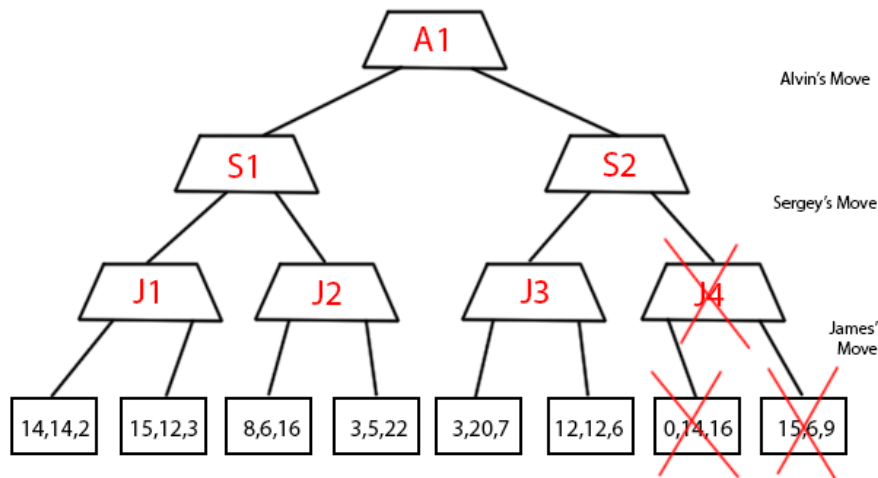
Three of your TAs, Alvin, Sergey, and James rent a cookie shuffler, which takes in a set number of cookies and groups them into 3 batches, one for each player. The cookie shuffler has three levers (with positions either UP or DOWN), which act to control how the cookies are distributed among the three players. Assume that 30 cookies are initially put into the shuffler.

Each player controls one lever, and they act in turn. Alvin goes first, followed by Sergey, and finally James. Assume that all players are able to calculate the payoffs for every player at the terminal nodes. Assume the payoffs at the leaves correspond to the number of cookies for each player in their corresponding turn order. Hence, an utility of (7,10,13) corresponds to Alvin getting 7 cookies, Sergey getting 10 cookies, and James getting 13 cookies. No cookies are lost in the process, so the sum of cookies of all three players must equal the number of cookies put into the shuffler. Players want to maximize their own number of cookies.

- (a) [3 pts] What is the utility triple propagated up to the root? **15,12,3**
- (b) [6 pts] Is pruning possible in this game? Fill in "Yes" or "No". If yes, cross out all nodes (both leaves and intermediate nodes) that get pruned. If no, explain in one sentence why pruning is not possible. Assume the tree traversal goes from left to right.

☒ Yes.

☐ No, Reasoning:



Left lowest subtree: James chooses the node (15,12,3) at node J1. This gets propagated up to S1. Sergey doesn't know what value he can get on his right child, so we explore that. Upon propagating (8,6,16) to J2, we must explore J2's right child since there could be a triple better for James' best option (greater than 16) and better than Sergey's best option. (greater than 12). (15,12,3) gets propagated up to A1.

Alvin might have a good option (greater than 15) in the right subtree, so we explore down. On the (3,20,7) node, we propagate this up to J3. We need to continue going down this path because Sergey doesn't know if he can get more than 20, and Alvin doesn't know if he can get more than 15 (A1's value). Hence, (12,12,6) is explored. (3,20,7) is propagated to S2. Now, we can guarantee that Sergey will prefer any cookie count over 20. But, because the sum of cookies must be 30, this means that Alvin can get no more than 10 cookies in the right subtree. Hence, we can immediately prune any children of S2.